How Temporal Works

An architecture guide to durable execution, with examples from the `temporal-orchestrator` JavaScript demo

1. The problem Temporal solves

Real business processes span many steps and many systems: reserve stock, charge a card, wait for a human, ship a parcel. Any step can fail, any process can crash, and some steps wait for minutes or days. Building that reliably by hand means queues, retry tables, cron jobs, state machines and a lot of reconciliation code.

Temporal offers **durable execution**: you write the process as ordinary code, and Temporal guarantees it runs to completion, resuming exactly where it left off after any failure. It does this by recording every meaningful step in an **event history** and rebuilding the program's state from that history whenever it needs to.

2. The big picture

A Temporal system has two halves that are deliberately kept apart:

- **The Temporal Service** (the server). It stores state, keeps timers, holds task queues, and decides what should happen next. **It never runs your code.**
- **Workers** (your processes). They contain your workflow and activity code, poll the service for work, execute it, and report back. You deploy and scale them like any other service.

Clients are anything that talks to the service to start workflows, send them messages or read their results: a web API, a CLI, a scheduled job.

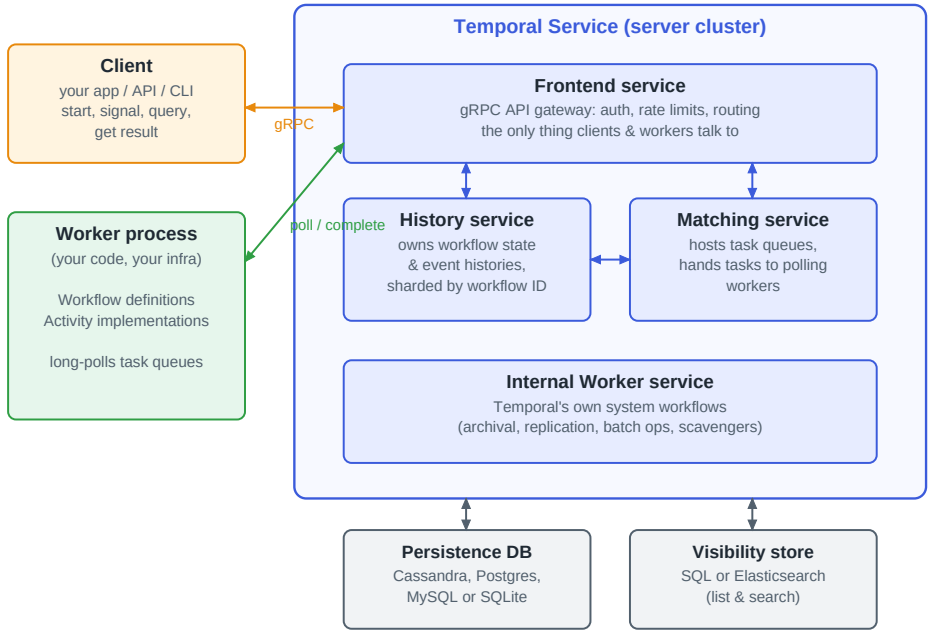

Figure 1. Components of a Temporal deployment. Arrows are gRPC calls; workers and clients only ever call the Frontend.

3. Inside the Temporal Service

The server is one binary made of four services. In the local dev server (`temporal server start-dev`) they all run in one process; in production each is scaled independently.

Service	Responsibility
Frontend	Stateless API gateway. Validates and rate-limits requests, then routes them to the History or Matching service that owns the relevant data.
History	The core. Holds each workflow's mutable state and append-only event history, runs durable timers, and decides which tasks to create. Work is split into a fixed number of **shards**; each workflow ID hashes to one shard, and each shard is owned by exactly one History host at a time. That single owner is what makes updates to a workflow consistent.
Matching	Hosts **task queues**. When History creates a task, Matching delivers it to a worker that is long-polling that queue (or stores it until one appears).
Internal Worker	Runs Temporal's own background workflows: archival, replication, batch operations, cleanup.

Storage

- **Persistence database** (Cassandra, PostgreSQL, MySQL or SQLite) is the source of truth for histories, mutable state, timers and task queues.
- **Visibility store** (SQL or Elasticsearch) indexes workflow metadata so you can list and search workflows ("all running orderWorkflow executions") without scanning histories.

Namespaces

A **namespace** is a unit of isolation: workflow IDs are unique per namespace, and retention, permissions and archival are set per namespace. The demo uses `default`.

4. Workers and task queues

A worker is a long-running process built with a Temporal SDK. On startup it registers the workflow and activity code it can run and begins **long-polling** one task queue. Workers make outbound calls only, so they can run behind a firewall with no open ports.

There are two kinds of task:

- **Workflow Task**: "advance this workflow". The worker runs the workflow code until it is blocked on something (an activity, a timer, a signal) and replies with a list of **commands** such as *ScheduleActivityTask*, *StartTimer* or *CompleteWorkflowExecution*.
- **Activity Task**: "run this function with these arguments". The worker executes it and reports success or failure.

Task queues are created on first use and act as built-in load balancers: start ten workers on the queue `orders` and tasks are spread across them. If every worker is down, tasks simply wait in the queue; nothing is lost.

5. Life of a workflow execution

This is what happens when the demo's client runs `client.workflow.start(orderWorkflow, ...)`:

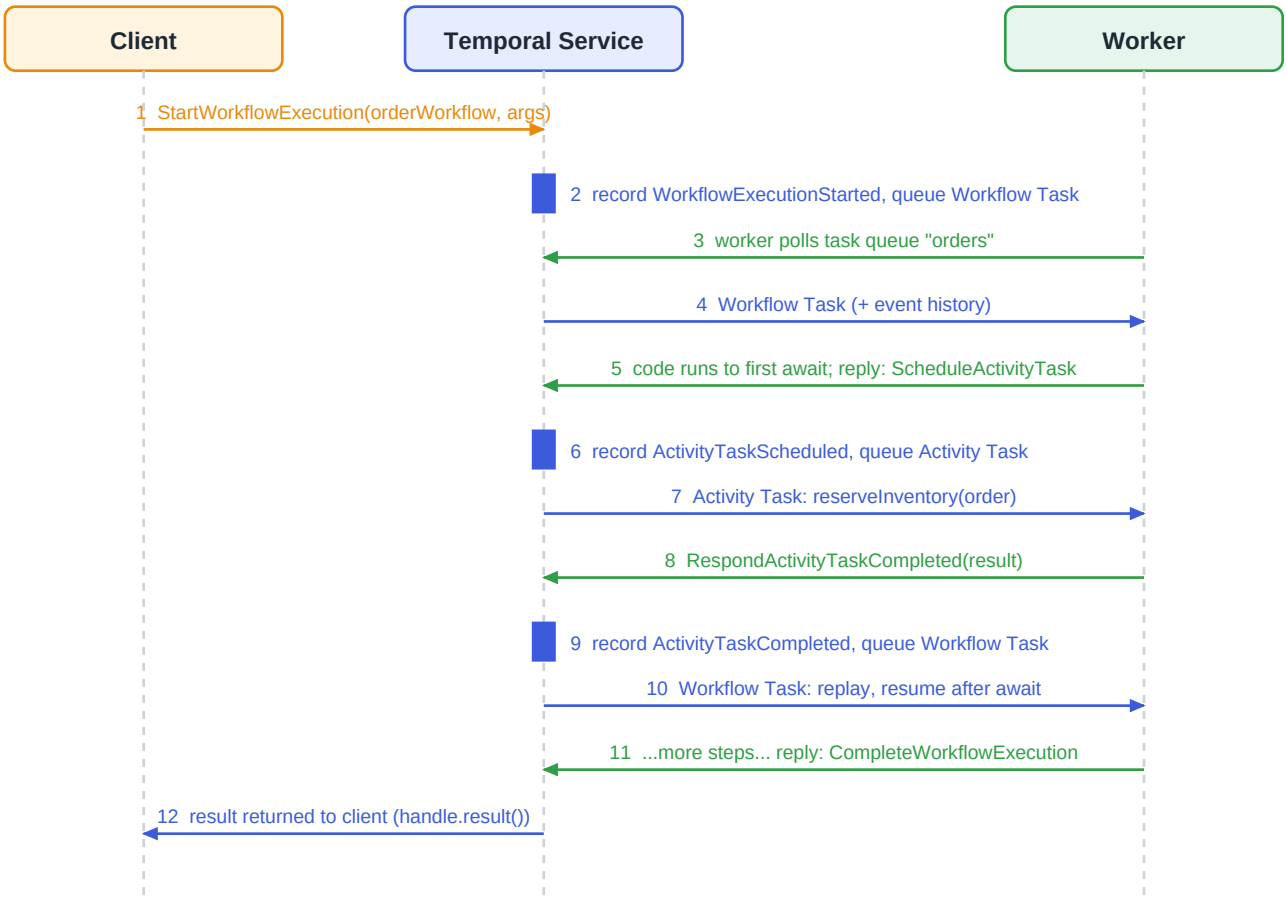

Figure 2. The conversation between client, service and worker. Every state change on the service side is written to the event history first.

The key point: the service drives the loop. The worker is stateless from Temporal's point of view, and any worker on the queue can process the next task, because the full history comes with it.

6. Event history and replay

Every workflow has an append-only **event history**. It is not a log for debugging; it **is** the workflow's state. A completed demo order produces events like these (simplified):

#	Event	Caused by in workflows.js
1	WorkflowExecutionStarted	client calls start()
5	ActivityTaskScheduled (reserveInventory)	await acts.reserveInventory(order)
7	ActivityTaskCompleted	activity returned rsv-order-...
11	ActivityTaskScheduled (chargePayment)	await acts.chargePayment(order)
13	ActivityTaskCompleted (attempt 3)	attempts 1-2 failed and were retried by the server
...	WorkflowExecutionSignaled (approve)	npm run approve
...	TimerStarted / TimerFired	await wf.sleep('5 seconds')
...	WorkflowExecutionCompleted	return { trackingNumber, ... }

When a worker crashes, or its cache evicts a workflow, the next worker rebuilds state by **replaying**: it runs the workflow function from the top, and every time the code reaches an await that already has an answer

in history, the SDK returns that recorded answer immediately instead of re-executing it. When replay reaches the end of history, the workflow is back to exactly where it was and continues live.

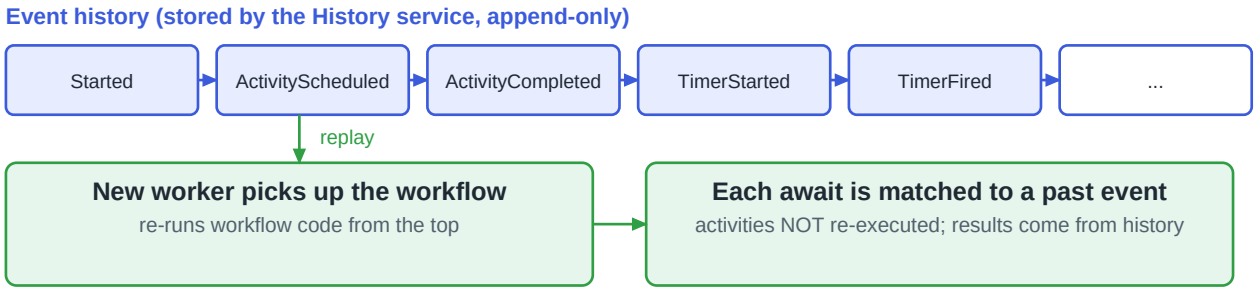

Figure 3. Recovery by replay. Side effects run once; their results are reused.

Why workflow code must be deterministic

Replay only works if the code makes the same decisions, in the same order, every time it runs with the same history. If it doesn't, the SDK raises a **non-determinism error**. So inside a workflow:

- No direct network, disk or database calls: put them in activities.
- No real clocks or random numbers. The TypeScript SDK runs workflows in a sandbox that replaces `Date` and `Math.random` with replay-safe versions.
- Use `wf.sleep`, not `setTimeout` from Node; use `wf.condition` to wait for state changes.
- Changing workflow logic while executions are running needs versioning (patching or worker versioning) so old histories still replay.

Activities have no such restriction. They are where all the messy, non-deterministic work lives.

7. Activities, retries and timeouts

When a workflow calls an activity, Temporal owns its reliability. The demo configures this in `proxyActivities`:

```
const acts = wf.proxyActivities({
  startToCloseTimeout: '10 seconds',   // max time for a single attempt
  retry: {
    initialInterval: '1 second',
    backoffCoefficient: 2,             // 1s, 2s, 4s, ...
    maximumAttempts: 5,
  },
});
```

Timeout	Meaning
Start-To-Close	Max duration of one attempt. Exceeded means the attempt failed and may be retried.
Schedule-To-Close	Max total time across all attempts.
Schedule-To-Start	Max time waiting in the queue for a worker (detects missing workers).
Heartbeat	For long activities: if the worker stops heartbeating, assume it died and retry elsewhere.

Retries are scheduled by the History service, not by your code, so they survive worker crashes. Errors thrown with `ApplicationFailure.nonRetryable(...)` skip retries, which is what the demo's card-limit rule does. Because an activity may run more than once, real activities should be **idempotent** (e.g. pass an idempotency key to the payment provider).

8. Talking to running workflows

Mechanism	Direction	Recorded in history?	Demo usage
Signal	client -> workflow, async, fire-and-forget	Yes	approve / reject
Query	client reads workflow state, read-only	No	status
Update	client -> workflow, synchronous, returns a result, can validate	Yes	(not used)

Signals are stored by the service even while no worker is running. That is why, in the demo, you can stop the worker, send `approve`, restart the worker and watch the order complete.

9. Durable timers

`await wf.sleep('30 days')` does not keep a process alive for 30 days. The worker sends a *StartTimer* command, the History service persists the timer, and when it fires a *TimerFired* event and a new workflow task are created. In the meantime the workflow uses no worker resources at all. `wf.condition(fn, '2 minutes')` in the demo combines a durable timer with a wait for a signal.

10. Putting it together: the demo

Concept	In the demo
Temporal Service	docker-compose.yml runs `temporal server start-dev` (all services + in-memory SQLite, UI on :8233)
Worker	src/worker.js polls task queue `orders`
Workflow	src/workflows.js: `orderWorkflow`
Activities	src/activities.js: reserve, charge (flaky), ship, plus refund/release for compensation
Client	src/client.js: start, signal, query
Saga / compensation	on failure the workflow runs recorded undo steps in reverse; these are just more activities, so they are retried and durable too

Key takeaways

- The service stores state and schedules work; your workers execute code. The two only meet through task queues.
- The event history is the workflow's state. Recovery is replay, so workflow code must be deterministic.
- Side effects live in activities, which Temporal times out and retries for you.
- Waiting (timers, signals) is free and durable, whether it lasts seconds or months.
- Further reading: docs.temporal.io (Temporal Service, Workflows, Activities, Task Queues, Message Passing, TypeScript SDK guide).